

Query Pricing Under Recomposition: An Experimental Evaluation of Arbitrage-Free Pricing Constructions

Zaheer Ahmad Gondal

School of Computer Science and Technology, Huazhong
University of Science and Technology
Wuhan, Hubei, China
zaheerahmad@hust.edu.cn

Ghalib Nadeem

School of Computer Science and Technology, Huazhong
University of Science and Technology
Wuhan, Hubei, China
ghalibnadeem@hust.edu.cn

Xiaofeng Ding

School of Computer Science and Technology, Huazhong
University of Science and Technology
Wuhan, Hubei, China
xfding@hust.edu.cn

Jamil Ahmad

School of Computer Science and Technology, Huazhong
University of Science and Technology
Wuhan, Hubei, China
I202522051@hust.edu.cn

Abstract

Arbitrage-free query pricing has produced a decade of constructions but no common yardstick. Each paper defines its own priced object and information semantics, making reported numbers incomparable. We build an evaluation substrate where eleven pricing constructions from three published systems price the same assets, face the same recomposing buyer, and take the same metrics. We evaluate them on a TPC-H instance with 368 assets and 1046 verified determinacy pairs, and on 975 hand-written SQLShare queries forming 15 markets with 2972 verified pairs. We verify every determinacy relation by re-execution and back every reported arbitrage with a re-run purchase plan (908 executed, discarded on failure). We report four measurements. Revenue and welfare order the constructions in opposite directions: uniform bundle pricing earns .558 and .558 of the per-asset monopoly benchmark while returning welfare of .413 and .413, compared to .557/.557 revenue and .432/.432 welfare for information-proportional constructions. Conflict-set containment falsified 0 of 047 claimed derivability edges on the largest real market, falsifying none on TPC-H. Support-set sampling as specified prices 83 of 222 assets at 7 distinct price levels, while readership weighting prices 165 at 39 levels for 82 times the build cost. Aggregate roll-up accounts for 0.8% of verified pairs in the real workload against 71.4% for projection. Two findings confirm published results on a new workload; one from a synthetic pilot failed to replicate.

PVLDB Reference Format:

Zaheer Ahmad Gondal, Xiaofeng Ding, Ghalib Nadeem, and Jamil Ahmad.
Query Pricing Under Recomposition:
An Experimental Evaluation of Arbitrage-Free Pricing Constructions.
PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at https://github.com/zaheer/VLDB_EAB_Artifact_-_Arbitrage-Free-Pricing-Constructions.

1 Introduction

A data seller who prices queries rather than whole datasets faces a buyer who can rewrite the request. If the answer to an expensive query can be reconstructed from cheaper ones, the posted price is fiction. The line of work starting with view-based query pricing [16, 18] and continuing through QIRANA [8, 9] and the arbitrage characterizations that follow [3, 21] answers this by constructing price functions that are arbitrage-free: no rewrite is cheaper than the direct purchase.

Each construction proves its guarantee. None has been measured against another. The reason is structural rather than sociological: each paper defines its own priced object, its own semantics for what an answer reveals, and its own class of buyer rewriting, so the numbers in one paper's tables are not comparable to the numbers in another's. The field has correctness proofs and no common yardstick.

This paper supplies the yardstick and reports what it measures. We build a substrate on which several published constructions can price the same assets, be audited by the same adversary, and be scored on the same revenue and coverage metrics. We then run four constructions from two published systems against five baselines on two workloads.

1.1 What the substrate has to get right

Three properties separate a fair comparison from a rigged one, and each costs real engineering.

We verify determinacy, never declare it. A structural rule proposes that one asset's answer can be derived from another; the harness performs the rewrite and accepts the pair only when the derived answer equals the independently evaluated answer. On TPC-H we tested every ordered pair at two scales, 54080 pairs in total, and measured candidate-rule precision and recall of 1.000. Verification's value here is not rejecting false positives, since there

were none. It is defect detection: it caught a date-type mismatch that would have made 124 of 172 rewrites silently return empty answers while a declaration-based harness reported them all as valid.

The adversary is a solver, not a heuristic. A buyer who recomposes solves a min-cost acquisition problem. We solve it exactly as an integer program over two decomposition families, and we re-execute every plan the solver returns and compare the reconstruction against the target answer. Across the experiments reported here, 908 plans were re-executed and none failed to reconstruct.

Ports are reported with their fidelity. Every mechanism in the comparison is a port of published work or a labeled baseline. None is ours. Where the substrate cannot support a construction exactly, we record the deviation and report it beside the result, so a weak number can be attributed to the port rather than to the mechanism. One such case is instructive: our first entropy port took the binary entropy of the distinguished fraction, which is not monotone, and produced a 22% monotonicity-violation rate that would have read as a finding about QIRANA. It was a defect in our port.

1.2 Contributions

- (1) A common evaluation substrate for query pricing with operationally verified determinacy, an exact multi-source acquisition adversary with plan verification, and per-asset provenance. It runs on generated benchmark workloads and on real query logs through the same interface (Section 3).
- (2) Ports of four pricing functions from two published systems, each implemented from its defining rule, with a fidelity table stating what is faithful and what deviates (Section 4).
- (3) A head-to-head evaluation on TPC-H and on 975 handwritten SQLShare queries, reporting revenue against non-strategic and recomposing buyers, two arbitrage audits, coverage, and stability across four valuation families (Sections 6 and 7).
- (4) Three findings about the published constructions: arbitrage-freeness is not what limits their revenue; the sampling that makes support-set pricing tractable does not inherit the arbitrage-freeness the construction proves; and the derivability relation the literature treats as canonical is the rarest one in a real workload.
- (5) A negative result, reported as such. Of three candidate findings from a synthetic pilot, one survived both workloads.

1.3 What this paper does not do

We propose no new algorithms or theoretical proofs; rather, our core contribution is the rigorous empirical measurement of existing mathematical guarantees under real-world constraints. Buyer valuations are synthetic, because no public dataset records what anyone would pay for a SQL answer; we treat the valuation family as a measured axis and report whether conclusions move with it (Section 8). Our planner searches a structured subspace of all valid decompositions, so the arbitrage gains we report are lower bounds. Determinacy is instance-level over a restricted fragment, not schema-level query determinacy, which is undecidable in general.

2 Background and Related Work

Data pricing covers several distinct problems that share a vocabulary: pricing answers to queries, pricing whole datasets, pricing trained models, valuing data for machine learning, and allocating revenue among providers. This paper addresses the first. Pei surveys the wider area and separates these threads [25]; we cite the others only where they bear on our substrate or our metrics.

2.1 Query pricing

Koutris et al. introduced query-based pricing and the arbitrage conditions this paper audits [15], developed the theory in its journal form [18], and built the QUERYMARKET prototype [16, 17]. A seller prices a set of views; the price of a query is the cost of the cheapest subset of views that determines it, found by an integer program. Two conditions constrain any price function. Information arbitrage: if Q_2 determines Q_1 , then $\pi(Q_1) \leq \pi(Q_2)$, since a buyer who wants Q_1 would otherwise buy Q_2 and derive it. Combination arbitrage: $\pi(Q_1 \parallel Q_2) \leq \pi(Q_1) + \pi(Q_2)$, since a buyer would otherwise split the request across two accounts. A price function monotone and subadditive in the appropriate order satisfies both.

Determinacy is undecidable in general, so Deep and Koutris replaced it with a tractable proxy in QIRANA [8, 9]. Fix a support set S of database instances. The conflict set $C_S(Q) = \{D' \in S : Q(D') \neq Q(D)\}$ collects the instances an answer rules out, and any monotone subadditive function of it is arbitrage-free, because determinacy implies $C_S(Q_1) \subseteq C_S(Q_2)$ for every S . Lin and Kifer characterized which pricing functions avoid arbitrage under more general conditions [21].

The line has since extended in three directions we do not evaluate but note for scope: to graph queries [4], to incomplete data where the answer itself is uncertain [22], and to online settings where valuations decay over time [10]. Scalability of the conflict-set computation remains an active concern [26].

2.2 Revenue, and what is already known

Chawla et al. asked what these constructions cost in revenue [3]. They reduce query pricing to revenue maximization with single-minded buyers and unlimited supply, restrict attention to tractable subclasses, and give approximation algorithms for each: uniform bundle pricing, uniform item pricing, an LP-based item pricing, a primal-dual capacity item pricing, and a greedy layering algorithm. They evaluate these on generated workloads over TPC-H and SSB, and report that the algorithms with the best worst-case bounds are not the best empirically.

That paper is the closest prior work to ours and we are explicit about the overlap. Two of our results confirm theirs rather than discover anything. First, the simplest construction wins on revenue: in our measurements uniform bundle pricing earns .558 of the per-asset monopoly benchmark on TPC-H and 0.609 on SQLShare, above every other algorithm on both workloads. Second, the ranking does not follow the approximation bounds. We report these as replications on a workload they did not use, and the correct reading of our flat-fee baseline is that it is their uniform bundle pricing; the two agree to within 2%.

Three things separate this paper from theirs. They evaluate revenue with valuations revealed to the seller, which is what their

algorithms assume; we hold out an independent buyer sample, and that is where the algorithms separate (Section 6). They compare algorithms within one framework; we compare across frameworks, which requires a substrate on which a view-cover price and a conflict-set price apply to the same assets. And they evaluate revenue; we measure buyer surplus alongside it, which reverses the ranking (Sections 6 and 7).

2.3 Model pricing and data valuation

Two adjacent problems share the pricing vocabulary and are not what we measure. Chen et al. price trained models rather than query answers, offering a menu of model instances at different noise levels [5, 6]; the priced object is a model, so arbitrage there concerns model accuracy rather than answer derivability. Data valuation for machine learning attributes a trained model’s performance back to training points, usually through the Shapley value or influence functions [7, 23, 27]. Because exact Shapley computation is exponential, much of that work is about approximation [11, 14, 29, 30]. An attribution is not a market price, and we keep the two apart: nothing in our evaluation uses a Shapley value, and none of the mechanisms we port claims to.

Incentive design for acquiring data is a third neighbour. Truthfulness becomes the central property when buyers or sellers report private values [31], and incentive schemes for collecting data from contributors have been studied directly [2]. Our setting is posted-price, so truthfulness is vacuous, which Section `refsec:setup` states rather than tests.

2.4 Determinacy, views, and provenance

Whether a query can be answered from a set of views is the classical problem underlying information arbitrage. Halevy surveys answering queries using views [13], Nash et al. study determinacy and rewriting directly [24], and Cao et al. give conditions under which bounded rewriting is possible [1]. These results are why every deployed pricing construction either restricts the view language or substitutes a proxy. We take the route available to an evaluation and not to a deployed system: propose a rewrite, execute it, and accept the pair only when the derived answer matches the independently evaluated one.

Our support-size cost model is computed from provenance rather than estimated [12], which matters because two of the ported mechanisms price in proportion to how much of the database an answer reads.

2.5 Adjacent measurement concerns

Three further lines bear on the substrate. Approximate query processing quantifies the cost of sampling-based answers [20], which is the cost model behind the support-set sampling we measure in Section 7. Query answering under differential privacy prices a different resource, the privacy budget [19], and the arbitrage conditions need reformulation there, so we exclude it. Ranking over uncertain and incomplete answers [28] is the setting that motivates pricing on incomplete data, which we also do not cover.

2.6 Evaluation practice

What the field does not have is a shared substrate on which two constructions price the same assets and face the same adversary. The consequence is visible in the primary sources: each reports the cost of computing its own prices, and none reports what a competing construction would have charged for the same query. To our knowledge, among the query-pricing constructions cited above, no published head-to-head measurement exists.

3 The Evaluation Substrate

A comparison across pricing frameworks needs one definition of the priced object, one determinacy relation, one adversary, and one cost model. This section states each and reports what it costs to establish.

3.1 Assets

An *asset* is a query paired with its answer on a fixed instance. The seller’s catalog is a set of assets $\mathcal{A}$, and a pricing mechanism is a map $\pi : \mathcal{A} \rightarrow \mathbb{R}_{\geq 0}$. Fixing the instance is what lets a view-cover price and a conflict-set price apply to the same object: the former needs a catalog of views, the latter needs answers, and an asset supplies both.

3.2 Determinacy, verified rather than declared

We write $a_2 \succeq a_1$ when a_1 ’s answer can be derived from a_2 ’s. A structural rule proposes a candidate, we execute the rewrite, and we compare the derived answer against the independently evaluated one. We never record a determinacy the harness has not run.

We measure the oracle’s own error rate rather than assuming it. On TPC-H we attempted reconstruction for every ordered pair of assets at two scales, 54080 pairs, and compared the candidate rule against the executed outcome. Precision and recall are both 1.000: the rule proposes no pair that fails, and misses none that succeeds.

Verification’s value on this workload is therefore not the rejection of false candidates, since there were none. It is defect detection. A date-type mismatch in an early version silently emptied the answer of every month-restricting rewrite; 124 of 172 candidates returned empty and matched nothing, while a declaration-based harness would have recorded all 172 as valid determinacies.

3.3 Instance-level derivability is weaker than determinacy

On the real workload the structural rules are answer-level: column containment, row containment, and aggregate roll-up, each checked on the instance. Such a check is sound for the instance and not for the schema. Determinacy quantifies over all instances, so an edge that holds on D need not hold on D' .

This gap is measurable, and Section 7 measures it. Because determinacy implies $C_S(a_1) \subseteq C_S(a_2)$ for every support set, any claimed edge whose conflict sets violate that containment is provably not a determinacy edge, and the distinguishing instance is the witness. Conflict-set sampling is therefore a sound falsifier for a derivability oracle, and we use it as one.

3.4 The adversary

A buyer who will not pay the posted price solves a min-cost acquisition problem: buy the cheapest set of assets from which the wanted answer can be rebuilt. We solve it exactly rather than heuristically, as an integer program over two decomposition families, selection covers and attribute covers rejoined on a key. Disjointness is load-bearing, since overlapping sources double-count rows, which makes the selection case an exact cover.

Every plan the solver returns is re-executed and the reconstruction compared against the target answer. A plan that fails verification is discarded, so a discarded plan lowers the reported arbitrage rather than inflating it. Across the experiments in this paper, 908 plans were re-executed and failed, all in the two mechanisms that price many assets at zero.

3.5 Cost model

We record three per-asset quantities, and different mechanisms use different ones: answer size in cells, support size in base rows read, computed exactly from provenance rather than estimated, and measured execution time. No mechanism sees a quantity another cannot.

4 Taxonomy of Ported Mechanisms and Baselines

We classify the existing landscape of query pricing into two primary families: information-proportional pricing (which relies on structural mathematical proxies like conflict sets or view covers) and revenue-maximizing pricing (which learns from buyer valuations). This taxonomy, alongside our baselines, is illustrated in Figure 1.

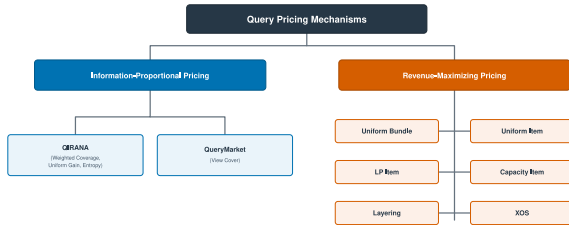


Figure 1: Taxonomy of evaluated query pricing mechanisms, categorized by their underlying mathematical models and design goals.

4.1 How the set was chosen

We included a construction when three conditions held: it is published as a query pricing mechanism, its price is defined by a rule we can execute rather than a property we would have to assume, and the substrate of Section 3 supplies its inputs. Three systems qualify, contributing eleven priced constructions in total.

Two candidates were considered and excluded, and we state why rather than omitting them silently. The instance-independent pricing of Lin and Kifer [21] is defined over a probability distribution on database instances; the substrate supplies a single instance and

a sampled neighbourhood, not a prior, so any port would price a different object and the comparison would measure our choice of prior. History-aware pricing with refunds prices a *sequence* of purchases, while every mechanism here prices a static catalog; including it would require a different protocol, not a different price function.

4.2 Provenance of each implementation

No public implementation of any of the three systems was located. We searched the publications, their demonstration papers, and the authors’ public repositories, and found prototypes described but no released code. Every mechanism below is therefore a reimplementa-tion from the paper’s defining rule, not a run of author code, and no result here should be read as a measurement of an original artifact. Where the substrate cannot supply a construction’s exact input we record the deviation in the fidelity table and report it beside the number.

4.3 Conflict-set pricing

Weighted coverage, uniform gain and entropy over the conflict set $C_S(a)$ [8]. Entropy is taken over the partition the answer induces on the support set, not over the distinguished fraction; the latter is not monotone, and an earlier version of our port used it and produced a 22% monotonicity violation rate that was a defect in the port rather than a property of the mechanism.

4.4 View-cover pricing

The price of an asset is the cost of the cheapest catalog subset that determines it [15, 18]. On TPC-H the catalog is the per-template, per-month view family, which is the construction’s native object. On SQLShare no such family exists, because the assets are queries users wrote rather than a generated lattice, so we substitute the maximal assets, those no other asset determines, as the catalog. That substitution is ours and is marked in every table.

4.5 Revenue-maximizing pricing

Five algorithms and their combination rule [3]: uniform bundle pricing, uniform item pricing, LP item pricing, capacity item pricing, the layering algorithm, and XOS. These tune themselves on the revealed training valuations, which is their design, so they receive no additional scale multiplier.

Two implementation choices matter for fidelity. The LP sweep must span the whole valuation range: restricted to the highest valuations it returns high-price, low-volume solutions and understates the algorithm, which cost us 15% of its revenue before we corrected it. And the layering algorithm requires a *minimal* cover, so that every edge in a layer holds an item no other edge holds; greedy coverage is not minimal, and we prune redundant edges to restore the property the unique-item pricing step depends on.

4.6 Baselines

Four reference points, labeled as baselines and not attributed to any paper: a uniform price, a price proportional to answer size, a price proportional to measured execution time, and the per-asset monopoly price fitted on the training sample, which is the normalizer for every revenue figure. A fifth, the monotone-constrained

optimizer, projects the monopoly prices onto the determinacy order by coordinate ascent and is reported as a fitted baseline, not as an optimum.

4.7 Fidelity

Table 1 states, per construction, its source, what the port implements faithfully, and where it deviates. A weak result for a construction with a recorded deviation should be attributed to the port.

5 Experimental Setup

5.1 Workloads and Dataset Justification

While algorithmic proposals often evaluate on numerous datasets, this benchmark prioritizes depth, structural complexity, and workload realism. We evaluate the mechanisms across two workloads that span the necessary spectrum for query pricing:

- **TPC-H (Scale Factor 1):** A standard synthetic benchmark providing a controlled, predictable view lattice. The catalog holds 368 assets over seven query templates, partitioned by month, yielding 1046 verified determinacy pairs.
- **SQLShare:** A massive, real-world query log containing 975 queries written by real users against their own uploaded tables. This forms 15 distinct markets carrying 2972 verified derivable pairs, providing the organic skew and complexity missing from generated benchmarks.

Together, these two datasets thoroughly stress-test both the mathematical assumptions of the pricing functions and their empirical scalability in the wild.

5.2 Compute Environment

All measurements were taken on a single machine (12 cores, 32 GB memory), single-threaded for the audits, using DuckDB for query execution and HiGHS for the integer programs. Runtime figures indicate relative costs between mechanisms on identical hardware.

5.3 Buyers

A buyer wants one asset and holds a private value for it. Demand over assets follows a skewed popularity distribution; the value is drawn log-normally around an information index of the wanted asset. We treat the valuation model as a swept axis rather than a fixed assumption, using four families: value proportional to answer cells, to support rows read, to rows times log support, and flat.

5.4 Fit, Freeze, Evaluate

Each mechanism sees a training sample of 800 buyers who pay posted prices, fits its one free parameter, and is then frozen. A second, independent sample of 800 buyers is drawn, and these buyers may recompose. Scale-fitted mechanisms receive one multiplier chosen on the training sample. The revenue-maximizing algorithms tune themselves on the training valuations as their source specifies, and receive no additional multiplier.

5.5 What is measured

We normalize revenue by the in-sample per-asset monopoly revenue. We record buyer surplus on the same scale and welfare is their sum. We report coverage for both buyer populations. Two

audits run against every price vector: the information-arbitrage audit checks $\pi(a_1) \leq \pi(a_2)$ over every verified determinacy pair, and the combination-arbitrage audit compares each asset’s price against the cost of the cheapest verified reconstruction plan.

5.6 Desiderata not measured

Individual rationality holds by construction of the settlement rule. Truthfulness is vacuous here: prices are posted, so there is nothing to misreport. Budget balance is not applicable to a single seller with unlimited supply.

6 Results: TPC-H

We price fifteen mechanisms (eleven ports, four baselines) over the TPC-H workload.

6.1 Revenue confirms the published result

Uniform bundle pricing earns .558 of the per-asset monopoly benchmark against recomposing buyers, the highest of any mechanism here, and our flat-fee baseline reaches .558, which is the same construction reached by a coarser fit. The three conflict-set pricing functions earn .426 to .557, a factor of 2.6 below. The LP, capacity and layering algorithms, which carry better worst-case guarantees than uniform bundle pricing, earn .543, .550 and .059.

The ordering does not follow the approximation bounds, which is the result Chawla et al. report and which we reproduce here rather than claim.

6.2 Welfare orders the mechanisms differently

Fig. 2(a) and Table 2 give revenue and welfare side by side. Size-proportional pricing and the conflict-set functions, the weakest mechanisms on revenue, are the strongest on welfare: .271 and .432 against .413 for uniform bundle pricing. Coverage explains the mechanism. The conflict-set functions serve 98% of buyers and uniform bundle pricing serves 46%.

A seller reading only the revenue column would discard exactly the constructions that move the most value.

6.3 Full information does not survive a held-out sample

Table 3 reports the fraction of training revenue lost out of sample. The revenue-maximizing algorithms assume valuations are revealed to the seller, and on this workload that assumption is expensive: LP item pricing loses 66.7%, layering gains SI48.1

percent, capacity item pricing 42.8%. Uniform bundle pricing, which fits a single scalar, loses 10.8%. The per-asset monopoly price, the most heavily fitted mechanism here, loses SI38.3 percent.

The pattern is a parameter count, not a property of revenue maximization as such: mechanisms that fit one number transfer, mechanisms that fit one number per support instance do not.

Table 1: Port fidelity. No original author implementation was located for any of the three systems, so every mechanism is a reimplementaion from the paper’s defining rule.

mechanism	source	deviation
QIRANA weighted coverage	Deep and Koutris, SIGMOD 2017	neighbours are deletions only, not the one-or-two-row modificati
QIRANA uniform gain	Deep and Koutris, SIGMOD 2017	as above
QIRANA entropy	Deep and Koutris, SIGMOD 2017	as above; entropy is taken over the distin- guished/undistinguishe
QUERYMARKET view cover [†]	Koutris et al., PODS 2012 and JACM 2015	explicit prices are placed on selection views only, which is the
uniform bundle (UBP)	Chawla et al., VLDB 2019, Section 5.1 (folk- lor	none
uniform item (UIP)	Chawla et al., VLDB 2019, Section 5.2, after G	none
LP item (LPIP)	Chawla et al., VLDB 2019, Section 5.2	the paper solves one LP per hyperedge; we cap the sweep at the K
capacity item (CIP)	Chawla et al., VLDB 2019, Section 5.2, after C	we solve the welfare LP and read item prices from the duals; the
layering	Chawla et al., VLDB 2019, Algorithm 1	minimal set cover is computed greedily, which the pa- per’s $O(Bm)$
XOS	Chawla et al., VLDB 2019, Section 5.2	inherits the deviations of its two inputs

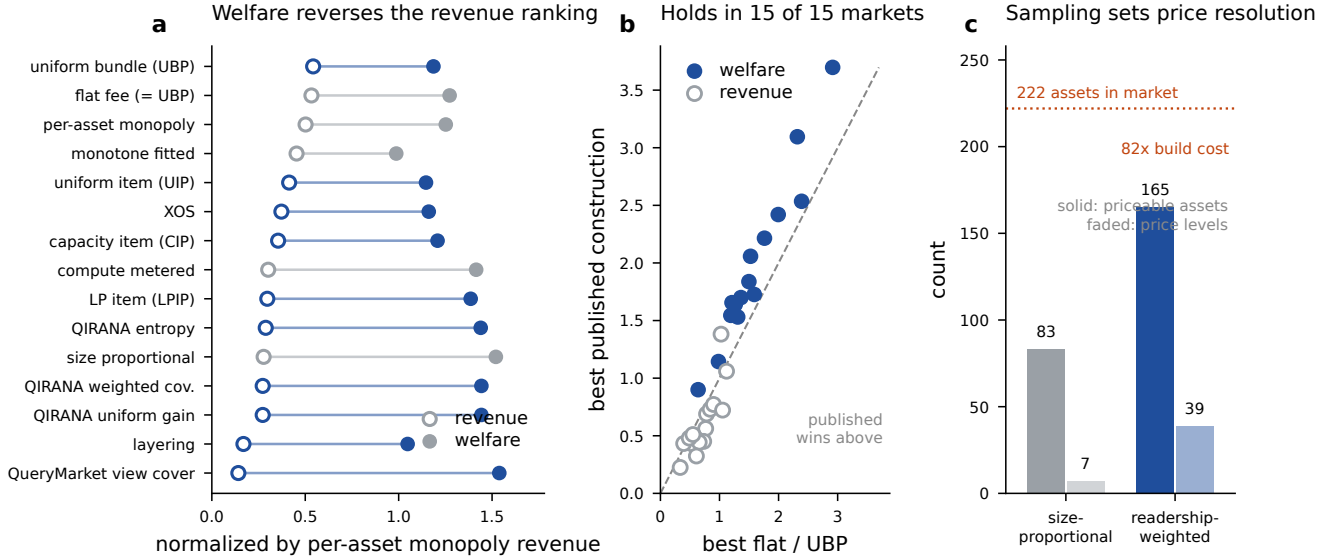


Figure 2: Published constructions against baselines on both workloads. (a) Revenue against recomposing buyers, normalized by the per-asset monopoly benchmark. (b) Price resolution of a sampled support set on the largest SQLShare market. (c) Arbitrage audits. Mean over four valuation families and five seeds.

6.4 Audits

Table 4 reports both audits. Every arbitrage-free construction scores zero on both, as its theorem requires, and this is the check that the ports are faithful rather than a finding. The fitted baselines do not: per-asset monopoly pricing inverts 22.4% of determinacy pairs and admits combination arbitrage on SI3.6

percent of assets, with a maximum gain of .841, meaning an asset reconstructible for

SI15.9

percent of its posted price.

Monotonicity is not subadditivity. The monotone-constrained baseline satisfies every determinacy-order constraint by construction and still admits combination arbitrage on

Table 2: Revenue, welfare and coverage against recomposing buyers, both workloads, normalized by the in-sample per-asset monopoly revenue. Mean over four valuation families and five seeds. Revenue and welfare order the mechanisms differently on both workloads. †: catalog substituted on SQLShare (Section 4).

mechanism	revenue		welfare		served	
	TPC-H	SQLShare	TPC-H	SQLShare	TPC-H	SQLShare
flat fee (= UBP)	0.466	0.600	1.150	1.396	0.52	0.37
size proportional	0.183	0.372	1.388	1.653	0.98	0.94
compute metered	0.359	0.245	1.130	1.700	0.58	0.92
per-asset monopoly	0.466	0.537	1.052	1.452	0.53	0.63
monotone fitted	0.473	0.434	0.881	1.093	0.43	0.26
QIRANA weighted coverage	0.181	0.363	1.376	1.509	0.98	0.60
QIRANA uniform gain	0.181	0.363	1.376	1.509	0.98	0.60
QIRANA entropy	0.208	0.368	1.368	1.511	0.97	0.61
QUERYMARKET view cover [†]	0.196	0.088	1.371	1.706	0.98	0.85
uniform bundle (UBP)	0.476	0.609	1.083	1.289	0.46	0.32
uniform item (UIP)	0.388	0.440	1.056	1.237	0.73	0.50
LP item (LPIP)	0.114	0.479	1.322	1.449	0.94	0.74
capacity item (CIP)	0.230	0.480	1.065	1.351	0.75	0.64
layering	0.119	0.219	1.048	1.048	0.82	0.65
XOS	0.230	0.514	1.053	1.269	0.74	0.55

Table 3: Generalization gap: fraction of training revenue lost on a held-out buyer sample. Positive is loss. The revenue-maximizing algorithms assume valuations are revealed to the seller, and that assumption is expensive on the generated workload and cheap on the real one.

mechanism	TPC-H	SQLShare
flat fee (= UBP)	+0.102	-0.108
size proportional	+0.072	-0.139
compute metered	+0.097	-0.410
per-asset monopoly	+0.537	+0.470
monotone fitted	+0.470	+0.411
QIRANA weighted coverage	+0.135	-0.060
QIRANA uniform gain	+0.135	-0.060
QIRANA entropy	+0.093	-0.067
QUERYMARKET view cover [†]	+0.083	+0.091
uniform bundle (UBP)	+0.108	-0.051
uniform item (UIP)	+0.163	+0.081
LP item (LPIP)	+0.667	+0.050
capacity item (CIP)	+0.428	+0.022
layering	+0.492	+0.067
XOS	+0.420	+0.000

SI1.8

percent of assets, with a maximum gain of .389. Enforcing the information-arbitrage condition alone does not close the combination channel.

7 Results: SQLShare

We evaluate the mechanisms across the 15 distinct markets of the SQLShare workload.

7.1 The lattice is denser in the wild, and a different shape

Derivability is not an artifact of generated view families. The real corpus carries 3.69 verified pairs per asset against 2.54 on the generated TPC-H family. The shape differs from the literature’s canonical example. Projection accounts for 71.4% of verified pairs and selection for 27.8%, while aggregate roll-up, the relation used to motivate arbitrage-freeness in the founding papers, accounts for 0.8%. A mechanism tuned for roll-up derivability is tuned for the rarest case.

7.2 Conflict-set sampling falsifies derivability claims

Determinacy implies $C_S(a_1) \subseteq C_S(a_2)$ for every support set, so a claimed edge whose conflict sets invert cannot be a determinacy edge. Our instance-level oracle produces edges that are sound on the instance and not over all instances, and the inversions locate them.

On the largest market, 30 of 2047 claimed edges invert under a 30000-row support set. We searched for explicit witnesses, neighbour databases that change the target’s answer but not the source’s, and reproduced 23 of the 30 in an independent 4000-neighbour sample. A second support set, sampled differently, falsified 13 further edges. The oracle’s false-edge rate on this market is therefore at least 43 of 2047, or 2.10%, and the figure is a lower bound that tightens with sampling. Across all 15 markets, 48 of 2974 pairs are falsified, concentrated in 4 markets.

On TPC-H, where the rewrites are generic and schema-level, no edge has been falsified. This is the practical difference between a derivability relation that lifts to determinacy and one that does not, and it is measurable rather than assumed.

Table 4: Arbitrage audits. Information arbitrage is the fraction of verified derivability pairs whose prices invert; combination arbitrage is the fraction of assets a verified reconstruction plan buys more cheaply. Every reported violation is backed by a re-executed plan.

mechanism	information arb.		combination arb.		max gain	
	TPC-H	SQLShare	TPC-H	SQLShare	TPC-H	SQLShare
flat fee (= UBP)	0.000	0.000	0.000	0.000	0.000	0.000
size proportional	0.000	0.000	0.000	0.000	0.000	0.000
compute metered	0.150	0.409	0.014	0.360	0.181	0.989
per-asset monopoly	0.224	0.336	0.168	0.349	0.915	0.958
monotone fitted	0.000	0.000	0.028	0.000	0.574	0.069
QIRANA weighted coverage	0.000	0.006	0.000	0.009	0.000	1.000
QIRANA uniform gain	0.000	0.006	0.000	0.009	0.000	1.000
QIRANA entropy	0.000	0.006	0.000	0.009	0.000	1.000
QUERYMARKET view cover [†]	0.000	0.001	0.000	0.000	0.000	0.000
uniform bundle (UBP)	0.000	0.000	0.000	0.000	0.000	0.000
uniform item (UIP)	0.000	0.006	0.000	0.009	0.000	1.000
LP item (LPIP)	0.000	0.002	0.000	0.005	0.000	0.500
capacity item (CIP)	0.000	0.006	0.000	0.004	0.000	0.900
layering	0.000	0.001	0.000	0.006	0.000	0.800
XOS	0.000	0.006	0.000	0.006	0.000	0.900

Table 5: Support-set sampling on the largest SQLShare market, $|S| = 1000$. Sampling proportional to table size, which is what uniform sampling over the database amounts to, is nearly uncorrelated with which tables assets read ($r = -0.088$).

support sampling	priceable assets	median $ C_S $	price levels	build (s)
size proportional (as specified)	83 / 222	0	7	25
readership weighted	165 / 222	78	39	2043

Table 6: Spread across all 15 SQLShare markets with at least eight priced assets, reduced protocol (no cover planner). The welfare ordering is reported with its dispersion rather than from a single market.

mechanism	revenue (mean $\pm$ sd)	welfare (mean $\pm$ sd)
flat fee (= UBP)	0.700 $\pm$ 0.341	1.597 $\pm$ 0.934
per-asset monopoly	0.728 $\pm$ 0.376	1.404 $\pm$ 0.743
QIRANA weighted coverage	0.244 $\pm$ 0.304	1.947 $\pm$ 1.020
QUERYMARKET view cover [†]	0.585 $\pm$ 0.385	1.884 $\pm$ 0.922
uniform bundle (UBP)	0.714 $\pm$ 0.361	1.476 $\pm$ 0.882
LP item (LPIP)	0.341 $\pm$ 0.391	1.840 $\pm$ 1.018

7.3 The support sampler decides whether prices exist at all

Conflict-set pricing needs assets to be distinguished by the support set. The specification samples neighbours uniformly over the database, which in a real market means proportional to table size, and table size is almost uncorrelated with how many assets read a table ($r = -0.088$ here). Table 5 reports the consequence: at $|S| = 1000$, size-proportional sampling prices 83 of 222 assets with 7 distinct price levels, while readership-weighted sampling prices 165 with 39 levels. The better sampler costs 82 times more to build, because each neighbour triggers far more query re-executions.

The residue is the operational failure mode. 57 assets still have empty conflict sets under the better sampler, so they are free, and a free asset is a free reconstruction source. That is the mechanism behind the maximum combination-arbitrage gain of 1.000 in Table 4: a target rebuilt at no cost. Across all markets the median share of zero-priced assets at $|S| = 300$ is 37.5%, and in one market it is every asset.

7.4 The welfare reversal is not a single-market artifact

Table 6 reports the spread over all 15 markets. The published information-proportional constructions beat the flat fee and uniform bundle pricing on welfare in 15 of 15 markets. Flat pricing beats them on revenue in 13 of 15; in the two exceptions the market is either entirely zero-priced or has no zero-priced assets at all, the two ends of the resolution spectrum.

Dispersion is large. Mean welfare for conflict-set pricing is 1.947 with a standard deviation of 1.020 across markets, so the direction of the effect replicates while its magnitude does not. We report both.

7.5 Where the two workloads disagree

Full-information revenue maximization transfers here and did not on TPC-H. LP item pricing loses 5.0% of training revenue out of sample on SQLShare against 66.7% on TPC-H, and capacity item pricing 2.2% against 42.8%. Any conclusion about the fragility of these algorithms that is drawn from a generated benchmark alone does not survive contact with a real workload.

8 Threats to Validity

8.1 Buyer valuations are synthetic

No public dataset records what a buyer pays for a SQL answer, and we did not find one. This is the largest threat to the revenue and welfare results, and it cannot be removed by better engineering. Our response is to treat the valuation model as a swept axis rather than a fixed assumption: four families, each with five seeds, with every result reported per family. We report a conclusion that holds in all four as holding, and one that moves as moving, with both numbers. Ranking stability is a measured quantity here, not a claim.

The finding most exposed to this threat is the revenue ordering, since revenue is a direct function of the value distribution. The findings least exposed are the structural ones: the rewrite-family distribution, the false-edge rate of the derivability oracle, and the price resolution of a sampled support set do not depend on buyer values at all.

8.2 Ports are reimplementations

No author code was located for any of the three systems, so every mechanism is our reimplementation from the paper’s defining rule. Section 4 reports two port defects we found and fixed; we cannot rule out defects we did not find. A construction that performs poorly here with a deviation recorded against it should be read as a statement about the port. The fidelity table exists so that a reader can make that judgement without re-deriving our code.

One deviation is load-bearing. The view-cover catalog on SQLShare is our construction, not the paper’s, because a hand-written query corpus has no generated view family. Its SQLShare numbers measure a substitution and are marked as such wherever they appear.

8.3 Determinacy is instance-level on the real workload

We check the SQLShare derivability relation by executing rewrites on one instance, which does not establish determinacy over all instances. We do not treat this as a caveat to be stated and moved past: Section 7 measures the resulting false-edge rate using conflict-set containment as a falsifier, and reports it as a lower bound that tightens with more sampling. The TPC-H relation uses generic schema-level rewrites and no edge there has been falsified.

8.4 Arbitrage gains are lower bounds

The acquisition planner searches two structured decomposition families rather than all valid rewritings, and plans that fail re-execution are discarded. Both choices push the reported arbitrage down. A mechanism reported here as arbitrage-free has survived this adversary, not every adversary.

8.5 Scope of the workloads

One generated benchmark and one real corpus, both relational and both SPJ with aggregation. Nothing here speaks to pricing over streams, graphs, or perturbed answers under a privacy budget, and the arbitrage conditions themselves need reformulation in the last case.

9 Conclusion

Four things a practitioner can act on, each tied to a table.

Do not evaluate a pricing scheme on revenue alone (Table 2). The published arbitrage-free constructions lose on revenue and win on welfare, on both workloads and in 15 of 15 markets. Which column a seller optimizes is a business decision, but reporting only one of them has caused a literature to look worse than it is.

Audit the sampler before the mechanism (Table 5). Conflict-set pricing degrades to giving assets away when the support set misses the tables buyers read, and uniform sampling over a real database does exactly that. Weighting by readership doubles the priceable assets and multiplies price resolution by more than five, at 82 times the build cost. Budget for it.

Treat a derivability oracle as a hypothesis (Section 7). Answer containment checked on one instance is not determinacy, and on real SQL it produced a false-edge rate of at least 2.10%. Conflict-set containment falsifies those edges for free, with witnesses, and any system that maintains a derivability index should run it.

Hold out the buyers (Table 3). Algorithms that fit one parameter per support instance lose up to SI38.3

percent of their revenue on an independent sample, and the loss is invisible in-sample. It is also workload-dependent: the same algorithms lose under 5% on the real corpus, so a generated benchmark alone will mislead in either direction.

We contribute no pricing mechanism. We release the substrate, the ports, the workloads and every result table so that the next construction can be measured against these rather than described alongside them.

Declarations

CRedit. Zaheer Ahmad Gondal (Lead): conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing (original draft and revision), visualization. Xiaofeng Ding (Supervisor): conceptualization, methodology, resources, supervision, writing (review and editing). Ghalib Nadeem (Supporting): software, validation, writing (review and editing). Jamil Ahmad (Supporting): validation, writing (review and editing).

Equal treatment. Every mechanism with a free scale

parameter received the same fitting budget on the same training sample; the

revenue-maximizing algorithms tune themselves on the training valuations, as their

source specifies, and received no additional multiplier.

Conflict of interest. None declared.

Ethics. No human subjects. The SQLShare corpus is a public release restricted to users who consented to publication; the artifact ships a download script rather than the data.

Funding. None declared.

Data availability. TPC-H is generated by the script in the artifact. SQLShare is downloaded by the script in the artifact from its public release. All result tables and the code that produced them are in the artifact.

Generative AI assistance. An AI assistant was used for code implementation, experiment execution, and drafting. All experimental design decisions, result interpretation and final text are the author's responsibility, and every reported number is generated from the saved result files by `tools/make_tables.py`.

References

- [1] Yang Cao, Wenfei Fan, Floris Geerts, and Ping Lu. 2018. Bounded Query Rewriting Using Views. *ACM Transactions on Database Systems* 43, 1 (2018), 1–46. doi:10.1145/3183673
- [2] Chengliang Chai, Ju Fan, and Guoliang Li. 2018. Incentive-Based Entity Collection Using Crowdsourcing. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 341–352. doi:10.1109/icde.2018.00039
- [3] Shuchi Chawla, Shaleen Deep, Paraschos Koutris, and Yifeng Teng. 2019. Revenue maximization for query pricing. *Proceedings of the VLDB Endowment* 13, 1 (2019), 1–14. doi:10.14778/3357377.3357378
- [4] Chen Chen, Ye Yuan, Zhenyu Went, Guoren Wang, and Anteng Li. 2022. GQP: A Framework for Scalable and Effective Graph Query-based Pricing. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 1573–1585. doi:10.1109/icde53745.2022.00163
- [5] Lingjiao Chen, Paraschos Koutris, and Arun Kumar. 2019. Towards Model-based Pricing for Machine Learning in a Data Marketplace. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, 1535–1552. doi:10.1145/3299869.3300078
- [6] Lingjiao Chen, Hongyi Wang, Leshang Chen, Paraschos Koutris, and Arun Kumar. 2019. Demonstration of Nimbus. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, 1885–1888. doi:10.1145/3299869.3320231
- [7] Sang Choe, Hwijee Ahn, Juhan Bae, Kewen Zhao, Youngseog Chung, Adithya Pratapa, Willie Neiswanger, Emma Strubell, Teruko Mitamura, Jeff Schneider, Eduard Hovy, Roger Grosse, and Eric Xing. 2025. What is Your Data Worth to GPT? LLM-Scale Data Valuation with Influence Functions. In *Advances in Neural Information Processing Systems 38*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 162015–162056. doi:10.52202/085713-4883
- [8] Shaleen Deep and Paraschos Koutris. 2017. QIRANA: A Framework for Scalable Query Pricing. In *Proceedings of the 2017 ACM International Conference on Management of Data*. ACM, 699–713. doi:10.1145/3035918.3064017
- [9] Shaleen Deep, Paraschos Koutris, and Yash Bidasaria. 2017. QIRANA demonstration. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1949–1952. doi:10.14778/3137765.3137816
- [10] Yicheng Fu, Xiaoye Miao, Huanhuan Peng, Chongning Na, Shuiguang Deng, and Jianwei Yin. 2024. Online Query-Based Data Pricing with Time-Discounting Valuations. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3449–3461. doi:10.1109/icde60146.2024.00266
- [11] Felipe Garrido-Lucero, Benjamin Heymann, Maxime Vono, Patrick Loiseau, and Vianney Perchet. 2024. DU-Shapley: A Shapley Value Proxy for Efficient Dataset Valuation. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 1973–2000. doi:10.52202/079017-0063
- [12] Todd J. Green and Val Tannen. 2017. The Semiring Framework for Database Provenance. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. ACM, 93–99. doi:10.1145/3034786.3056125
- [13] Alon Y. Halevy. 2001. Answering queries using views: A survey. *The VLDB Journal* 10, 4 (2001), 270–294. doi:10.1007/s007780100054
- [14] Jingxuan He, Changshuo Liu, Shaofeng Cai, Xixian Han, Yanyan Shen, and Beng Chin Ooi. 2026. CoShap: A Scalable Coalition Growth Approach to Shapley Value Approximation. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–27. doi:10.1145/3802032
- [15] Paraschos Koutris, Prasang Upadhyaya, Magdalena Balazinska, Bill Howe, and Dan Suciu. 2012. Query-based data pricing. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI symposium on Principles of Database Systems*. ACM, 167–178. doi:10.1145/2213556.2213582
- [16] Paraschos Koutris, Prasang Upadhyaya, Magdalena Balazinska, Bill Howe, and Dan Suciu. 2012. QueryMarket Demonstration: Pricing for Online Data Markets. *Proceedings of the VLDB Endowment* 5, 12 (2012), 1962–1965. doi:10.14778/2367502.2367548
- [17] Paraschos Koutris, Prasang Upadhyaya, Magdalena Balazinska, Bill Howe, and Dan Suciu. 2013. Toward practical query pricing with QueryMarket. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. ACM, 613–624. doi:10.1145/2463676.2465335
- [18] Paraschos Koutris, Prasang Upadhyaya, Magdalena Balazinska, Bill Howe, and Dan Suciu. 2015. Query-Based Data Pricing. *J. ACM* 62, 5 (2015), 1–44. doi:10.1145/2770870
- [19] Chao Li and Jerome Miklau. 2012. An adaptive mechanism for accurate query answering under differential privacy. *Proceedings of the VLDB Endowment* 5, 6 (2012), 514–525. doi:10.14778/2168651.2168653
- [20] Xi Liang, Stavros Sintos, Zechao Shang, and Sanjay Krishnan. 2021. Combining Aggregation and Sampling (Nearly) Optimally for Approximate Query Processing. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, 1129–1141. doi:10.1145/3448016.3457277
- [21] Bing-Rong Lin and Daniel Kifer. 2014. On arbitrage-free pricing for general data queries. *Proceedings of the VLDB Endowment* 7, 9 (2014), 757–768. doi:10.14778/2732939.2732948
- [22] Xiaoye Miao, Yunjun Gao, Lu Chen, Huanhuan Peng, Jianwei Yin, and Qing Li. 2022. Towards Query Pricing on Incomplete Data. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2022), 4024–4036. doi:10.1109/tkde.2020.3026031
- [23] Bruno Mlodozieniec, Isaac Reid, Sam Power, David Krueger, Murat Erdogdu, Richard Turner, and Roger Grosse. 2025. Distributional Training Data Attribution: What do Influence Functions Sample?. In *Advances in Neural Information Processing Systems 38*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 11110–11144. doi:10.52202/085713-0339
- [24] Alan Nash, Luc Segoufin, and Victor Vianu. 2010. Views and queries. *ACM Transactions on Database Systems* 35, 3 (2010), 1–41. doi:10.1145/1806907.1806913
- [25] Jian Pei. 2022. A Survey on Data Pricing: From Economics to Data Science. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2022), 4586–4608. doi:10.1109/tkde.2020.3045927
- [26] Huanhuan Peng, Xiaoye Miao, Yicheng Fu, Jinshan Zhang, Shuiguang Deng, and Jianwei Yin. 2025. On Scalable Query Pricing in Data Marketplaces. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3140–3152. doi:10.1109/icde65448.2025.00235
- [27] Stephanie Schoch, Haifeng Xu, and Yangfeng Ji. 2022. CS-Shapley: Class-Wise Shapley Values for Data Valuation in Classification. In *Advances in Neural Information Processing Systems 35*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 34574–34585. doi:10.52202/068431-2505
- [28] Mohamed A. Soliman, Ihab F. Ilyas, and Shalev Ben-David. 2010. Supporting ranking queries on uncertain and incomplete data. *The VLDB Journal* 19, 4 (2010), 477–501. doi:10.1007/s00778-009-0176-8
- [29] Qiheng Sun, Jiayao Zhang, Jinfei Liu, Li Xiong, Jian Pei, and Kui Ren. 2024. Shapley Value Approximation Based on Complementary Contribution. *IEEE Transactions on Knowledge and Data Engineering* 36, 12 (2024), 9263–9281. doi:10.1109/tkde.2024.3438213
- [30] Jiayao Zhang, Qiheng Sun, Jinfei Liu, Li Xiong, Jian Pei, and Kui Ren. 2023. Efficient Sampling Approaches to Shapley Value Approximation. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–24. doi:10.1145/3588728
- [31] Liyang Zhu, Amina Manseur, Meng Ding, Jinyan Liu, Jinhui Xu, and Di Wang. 2024. Truthful High Dimensional Sparse Linear Regression. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 66995–67033. doi:10.52202/079017-2139